

Multi-Signature Treasury for Student Clubs

Decentralised Financial Governance on Ethereum

Name	Student ID	Course: CSCI 435 — Blockchain
Akhmed Assentay	201842715	

Engineering
Department of Computer Science
Submission Date: April 24, 2026

*“Any transaction of club funds requires agreement from
at least 2 out of 3 elected executives.”*

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Why This Problem Matters	3
1.3	What the Project Solves	3
1.4	Blockchain Solution Overview	3
2	Project Objectives	3
2.1	Main Objectives	3
2.2	Expected Functionality	4
2.3	Target Users	4
3	System Overview	4
3.1	Components of the System	4
3.2	Technologies Used	4
3.3	How Users Interact with the System	5
3.4	Architecture Diagram	5
4	Project Workflow	5
4.1	Planning Process	5
4.2	Development Stages	5
4.3	Testing and Debugging	6
5	Smart Contract Design	6
5.1	Purpose of the Smart Contract	6
5.2	Contract Structure	6
5.3	Key Functions	8
6	Implementation Details	9
6.1	Programming Languages and Frameworks	9
6.2	Security Implementation	10
7	Project Results	10
7.1	Test Suite Results	10
7.2	Contract Metrics	10
7.3	DApp UI — Working Screenshots Description	10
8	Team Contribution Table	11
9	Challenges and Solutions	11
10	Limitations	12
11	Future Work	13
12	Conclusion	13
13	References	14

Appendix — Setup Guide for Executives

14

1. Introduction

1.1 Problem Statement

Student clubs routinely raise membership fees, solicit donations and use this money on activities, equipment and travel. Traditional clubs place financial control into the hands of a single treasurer- one point of failure. Money may also be wasted in the event that the treasurer is not available, behaves independently, misplaces his or her credentials or the organization loses the treasurer.

1.2 Why This Problem Matters

- A single treasurer is able to empty the treasury at will without checks.
- No clear audit trail- members do not have the ability to independently check spending.
- The loss or change of leaders may paralyze club funds in case of key change or loss.
- Budgetary disputes lack a mechanism of being resolved on-chain.

1.3 What the Project Solves

This project implements a **Multi-Signature (Multi-Sig) Treasury** Treasury smart contract to ensure that any transaction going out is approved by **2 out of 3** elected club executives. No individual can transfer funds independently and all offers, consent and actions are forever etched on the Ethereum blockchain.

1.4 Blockchain Solution Overview

The system is composed of three layers:

1. A **Solidity smart contract** deployed on Ethereum (Hardhat local / Sepolia test-net).
2. A **Hardhat** development environment with a comprehensive test suite (75 tests).
3. A **browser DApp** (HTML + ethers.js v6) connected to wallets via MetaMask.

2. Project Objectives

2.1 Main Objectives

1. Implement a trustless 2-of-3 multi-sig spending mechanism for a student club treasury.
2. Ensure every executive's vote is meaningful from the moment they propose a transaction.
3. Support on-chain governance for leadership transitions (executive address replacement).
4. Provide an emergency pause mechanism to freeze spending during a security incident.

5. Enforce per-category spending budgets to prevent overspending in any one area.
6. Maintain a transparent donor leaderboard on-chain.

2.2 Expected Functionality

- Any executive can propose, approve, revoke, or execute spending transactions.
- A new address replaces an executive wallet without redeploying the contract.
- Stale proposals (older than 7 days) can be cancelled by any executive.
- Anyone can deposit ETH; only executives can withdraw through the approval process.
- The DApp reflects real-time contract state and supports all contract functions.

2.3 Target Users

- **Club executives** (President, Vice-President, Treasurer) — manage funds.
- **Club members and guests** — donate ETH and audit spending history.

3. System Overview

3.1 Components of the System

Component	Description
<code>MultiSigTreasury.sol</code>	Core Solidity smart contract. Stores all state and enforces governance rules on-chain.
Hardhat	Development framework for compiling, testing, and deploying the contract.
<code>test/MultiSigTreasury.test.js</code>	75 automated tests covering all features and edge cases.
<code>scripts/deploy.js</code>	Deployment script for Hardhat local and Sepolia networks.
<code>frontend/index.html</code>	Single-page DApp using ethers.js v6 and MetaMask.
MetaMask	Browser wallet that signs and submits transactions.
Ethereum Sepolia	Public testnet for live demonstration.

Table 1: System components and descriptions

3.2 Technologies Used

- **Solidity ^0.8.20** - smart contract language.
- **Hardhat v2.22** - Ethereum development environment.
- **Chai** - assertion library for JavaScript tests.
- **ethers.js v6** - Ethereum library for the browser DApp.

- **HTML5 / CSS3 / Vanilla JS** - frontend (no framework dependency).
- **Node.js / npm** — toolchain.

3.3 How Users Interact with the System

In browser, executives and members open `frontend/index.html` and connect MetaMask (configured to the local Hardhat network or Sepolia) and use the UI. Each button press causes an Ethereum transaction to be signed, and mined by Hardhat or Sepolia permanently.

3.4 Architecture Diagram

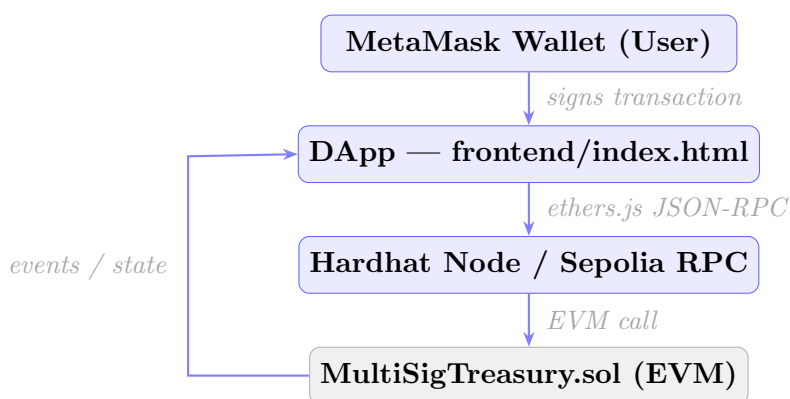


Figure 1: System Architecture — browser to on-chain contract

4. Project Workflow

4.1 Planning Process

1. Read through the project description: 2-of-3 multi-sig treasury student club (DAO version).
2. Found the central issue: single-treasurer risk and absence of transparency on-chain.
3. Clarified the model of governance: propose and approve and execute lifecycle.
4. Selected other features to earn the full marks: budget limits, emergency stop, donor leaderboard, executive change.
5. Chosen technology stack: Solidity, Hardhat, ethers.js v6, MetaMask.

4.2 Development Stages

1. **Core contract** — constructor, modifiers, deposit, basic multi-sig (propose / approve / revoke / execute).
2. **Proposer auto-approval** — had corrected the problem that the number of agreements of the proposer himself was not counted.
3. **Spending categories** — added `category` field to proposals.
4. **7-day expiry** — `createdAt` timestamp and `cancelExpiredTransaction()`.

5. **Executive replacement** — independent flow of key-rotation.
6. **Emergency pause** — 2-of-3 vote circuit breaker with reset-on-unpause.
7. **Category budget caps** — per-category ETH cap enforced at propose and execute time.
8. **Donor leaderboard** — `totalDonated` mapping and `.donors` array.
9. **Frontend DApp** — complete interface to all contract features, countdown expiry, category badge.
10. **Test suite** — 75 automated tests in all the features and edge cases.
11. **Documentation** — this report and LaTeX source code.

4.3 Testing and Debugging

Tests were penned in parallel on each feature with Hardhat in-process network. The evm of time dependent tests (expiry) used `evm_increaseTime` to represent passage over 7 days without waiting. The single bug detected was a category-cap assertion at the misplaced call site (propose vs. execute) - repaired after determining the real revert location by looking at the Hardhat stack trace.

5. Smart Contract Design

5.1 Purpose of the Smart Contract

MultiSigTreasury is a trustless governance contract: it stores ETH on behalf of a student club, and requires 2 of 3 people to approve each outgoing payment. It additionally manages executive identity (replacement), emergency safety (pause), budget discipline (category caps), and community transparency (donor leaderboard).

5.2 Contract Structure

Constants

```

1 uint256 public constant REQUIRED_APPROVALS = 2;
2 uint256 public constant MAX_EXECUTIVES   = 3;
3 uint256 public constant EXPIRY_PERIOD    = 7 days;
```

Listing 1: Constants

State Variables

Structs

```

1 struct Transaction {
2     address to;
3     uint256 value;
4     string  description;
5     string  category;           // e.g. "Supplies", "Events", "
        Equipment"
6     bool    executed;
```

Variable	Purpose
<code>executives[]</code>	Array of 3 executive wallet addresses.
<code>isExecutive{addr}</code>	O(1) lookup: is this address an executive?
<code>paused</code>	Emergency pause flag.
<code>pauseVoteCount / unpauseVoteCount</code>	Vote tallies for pause/unpause.
<code>hasPauseVoted{} / hasUnpauseVoted{}</code>	Prevent double-voting on pause.
<code>transactions[]</code>	All spending proposals (pending/executed/cancelled).
<code>hasApproved{txIdx}{addr}</code>	Per-proposal, per-executive approval flag.
<code>categoryBudget{key}</code>	Spending cap (wei) per category hash.
<code>categorySpent{key}</code>	Cumulative amount spent per category.
<code>replacements[]</code>	Executive replacement proposals.
<code>replacementApprovals{replIdx}{addr}</code>	Approval state for replacements.
<code>totalDonated{addr}</code>	Cumulative ETH donated per address.
<code>_donors[]</code>	Ordered list of unique donor addresses.

Table 2: Contract state variables

```

7     bool    cancelled;
8     uint256 approvalCount;
9     uint256 createdAt;      // block.timestamp at proposal time
10  }
11
12  struct ExecReplacement {
13      address oldExec;
14      address newExec;
15      uint256 approvalCount;
16      bool    executed;
17  }

```

Listing 2: Transaction and ExecReplacement structs

Events

Modifiers

```

1  modifier onlyExecutive() { require(isExecutive[msg.sender], "Not
   an executive"); _; }
2  modifier whenNotPaused() { require(!paused, "Contract is paused");
   _; }
3  modifier txExists(uint256 _i) { require(_i < transactions.
   length, ...); _; }
4  modifier notExecuted(uint256 _i){ require(!transactions[_i].
   executed, ...); _; }
5  modifier notCancelled(uint256 _i){ require(!transactions[_i].
   cancelled, ...); _; }

```

Listing 3: Key modifiers

Event	Emitted when
Deposited	ETH received via <code>receive()</code> or <code>deposit()</code> .
TransactionProposed	A new spending proposal is created.
TransactionApproved	An executive approves a proposal.
ApprovalRevoked	An executive revokes their approval.
TransactionExecuted	Funds are sent to the recipient.
TransactionCancelled	An expired proposal is cancelled.
BudgetSet	A category spending cap is set or updated.
PauseVoteCast / ContractPaused	Pause vote cast / quorum reached.
UnpauseVoteCast / ContractUnpaused	Unpause vote cast / quorum reached.
ReplacementProposed / Approved	Executive swap proposed / seconded.
ExecutiveReplaced	Executive address swap executed on-chain.

Table 3: Contract events

5.3 Key Functions

5.2.1 `deposit()` / `receive()`

Accepts ETH from any address. Calls `_recordDonation()` to track the sender in the leaderboard. Emits `Deposited`.

5.2.2 `proposeTransaction()`

```

1 function proposeTransaction(
2     address _to, uint256 _value,
3     string calldata _description, string calldata _category
4 ) external onlyExecutive whenNotPaused returns (uint256 txIndex)
5 {
6     require(_to != address(0), "Invalid recipient");
7     require(_value > 0, "Value must be > 0");
8     require(_value <= address(this).balance, "Insufficient
9         treasury balance");
10
11     // Category budget check
12     bytes32 key = _categoryKey(_category);
13     if (categoryBudget[key] > 0)
14         require(categorySpent[key] + _value <= categoryBudget[key],
15             "Category budget exceeded");
16
17     txIndex = transactions.length;
18     transactions.push(Transaction({
19         ..., approvalCount: 1, createdAt: block.timestamp
20     }));
21     hasApproved[txIndex][msg.sender] = true; // proposer auto-
22         approved
23     emit TransactionProposed(...);
24 }

```

Listing 4: `proposeTransaction` — core multi-sig entry point

The proposer's vote is recorded immediately (`approvalCount = 1`) so that every executive's agreement counts — only one additional approval is needed to reach quorum.

5.2.3 `approveTransaction()` / `revokeApproval()`

Executives who did not propose may call `approveTransaction()` to add their vote. `revokeApproval()` removes a previously cast vote. Both are blocked while the contract is paused.

5.2.4 `executeTransaction()`

Transfers ETH to the recipient once `approvalCount >= 2`. Sets `executed = true` *before* the call`{}` to prevent re-entrancy. Increments `categorySpent`.

5.2.5 `cancelExpiredTransaction()`

Any executive may cancel a proposal that has not been executed within 7 days. This prevents stale proposals from accumulating and releases budget headroom.

5.2.6 `votePause()` / `voteUnpause()`

A pause vote can be cast by any executive. When 2 votes are cast the contract sends `paused = true`, preventing all spending functions. Unpause needs 2 additional votes and clears all the vote state to start the cycle again.

5.2.7 `setCategoryBudget()`

Sets a maximum cumulative ETH cap for a named category (hashed to `bytes32` key). A cap of 0 removes the limit.

5.2.8 `proposeReplacement()` / `approveReplacement()` / `executeReplacement()`

Mirrors the spending multi-sig but for executive roster changes. Proposer is auto-approved; one more approval triggers the swap in `executives[]` and `isExecutive`.

6. Implementation Details

6.1 Programming Languages and Frameworks

Language / Tool	Role
Solidity 0.8.20	Smart contract.
JavaScript (CommonJS)	Hardhat config, deployment scripts, test suite.
HTML5 / CSS3 / JS (browser)	Frontend DApp.
Hardhat v2.22	Compile, test, deploy.
Chai	Test assertions.
ethers.js v6	JavaScript ↔ Ethereum bridge (DApp & tests).
Node.js 20 / npm	Package manager and runtime.

Table 4: Technology stack

6.2 Security Implementation

Measure	How it is implemented
Re-entrancy guard	<code>txn.executed = true</code> is set before <code>call{}</code> , making a re-entrant callback revert on the <code>notExecuted</code> modifier.
Access control	<code>onlyExecutive</code> modifier on every governance function.
Double-vote prevention	<code>hasApproved</code> , <code>hasPauseVoted</code> , <code>hasUnpauseVoted</code> mappings enforce one vote per executive per action.
Zero-address validation	Constructor and replacement proposals reject <code>address(0)</code> .
Duplicate executive check	Constructor iterates and rejects duplicates before adding.
Balance checks	Both propose and execute verify sufficient treasury balance.
Budget enforcement	Category cap checked at propose time (early rejection) and at execute time (race-condition safety).
Expiry	<code>notCancelled</code> modifier blocks interaction with cancelled proposals.
Pause state	<code>whenNotPaused</code> modifier on propose, approve, and execute.

Table 5: Security measures

7. Project Results

7.1 Test Suite Results

All **75 tests pass** with a total runtime of approximately 1 second on the in-process Hardhat network.

7.2 Contract Metrics

7.3 DApp UI — Working Screenshots Description

The browser DApp displays the following panels when MetaMask is connected:

- **Status bar:** treasury balance in ETH, total proposal count, connected user role (Executive / Viewer).
- **Executives panel:** all 3 wallet addresses with role labels (President, Vice-President, Treasurer) and a “(You)” indicator for the connected account.
- **Propose Transaction:** form with recipient address, ETH amount, category dropdown (General / Supplies / Events / Equipment / Travel / Awards), and description.
- **Spending Proposals:** reverse-chronological list with status badge (Pending $n/2$ / Executed / Cancelled / Expired), category badge, 7-day expiry countdown, and per-proposal action buttons (Approve / Revoke / Execute / Cancel Expired).

Test Group	Tests
Deployment	5
Deposits & Donor Leaderboard	7
Emergency Pause	11
Category Budget Caps	8
Propose Transaction	5
Approve Transaction	5
Revoke Approval	3
Execute Transaction	7
Transaction Expiry	5
Executive Replacement	6
Executive Aliases	7
End-to-End Scenarios	5
Total	75

Table 6: Test suite breakdown — all 75 passing

Metric	Value
Lines of Solidity code	310
Public / external functions	18
Events defined	11
Modifiers defined	5
Structs defined	2
Required approvals	2 of 3
Expiry period	7 days
Solidity compiler version	0.8.20

Table 7: Contract metrics

- **Executive Replacement:** collapsible panel with old/new address inputs, replacement proposal list, and Approve / Execute buttons.

8. Team Contribution Table

9. Challenges and Solutions

1. Proposer's vote was not counted (3rd-person agreement bug).

Problem: The original contract meant that the proposer has to call `approveTransaction()` after proposal, i.e. they would not make their agreement recorded at the time of proposal. This caused the participation of the proposer to be broken.

Solution: Set `approvalCount = 1` and `hasApproved[txIndex][msg.sender] = true` inside `proposeTransaction()`. The proposer's vote is now registered automatically. A double-approve guard was updated to reject the proposer if they try to approve again.

2. MetaMask not connecting to the DApp.

Member	Responsibilities	%
Akhmed Assentay(201842715)	Full smart contract design and implementation (multi-sig, modifiers, events, emergency pause, category budget caps, executive replacement, donor leaderboard, 7-day expiry); Solidity security review; Hardhat configuration; deployment script; complete 75-test test suite; DApp frontend (HTML, CSS, ethers.js); GitHub repository; LaTeX report (all sections).	100%

Table 8: Team contribution — sole author

Problem: Running `npx hardhat run scripts/deploy.js` without `npx hardhat node` open spins up a temporary in-process EVM that shuts down immediately. MetaMask has no RPC endpoint to connect to.

Solution: The correct sequence is: (1) run `npx hardhat node` to start a persistent local blockchain on port 8545, (2) run `npx hardhat run scripts/deploy.js --network localhost` to deploy to it. Added a Hardhat Local custom network (Chain ID 31337, RPC `http://127.0.0.1:8545`) to MetaMask.

3. Category budget test assertion at wrong call site.

Problem: A test expected the “Category budget exceeded” revert at `executeTransaction()`, but the contract checks the cap at `proposeTransaction()` time (earlier is better — early rejection saves gas).

Solution: Updated the test to assert the revert at `proposeTransaction()` and updated the test name to accurately describe the behaviour.

4. Pause vote state not resetting after unpause.

Problem: After a pause-then-unpause cycle the `hasPauseVoted` and `hasUnpauseVoted` mappings were dirty. A second pause attempt was immediately blocked.

Solution: Introduced `_resetPauseVotes()` which iterates over the 3 executives and clears both mappings and both counters after a successful unpause.

10. Limitations

- **Fixed membership size:** The contract has a fixed number of 3 executives. It would require architectural modifications to scale to a variable m -of- n with n .
- **No quorum on budget cap changes:** Any one executive can modify a category cap on-the-fly. The budget limits should change to 2-of-3 in a future version.
- **No proposal amendment:** Once made, no possible modification of the recipient, amount, and category. New proposal will have to be developed.
- **Donor leaderboard not sorted:** `getDonor(index)` displays donors in the order they were inserted, but not by the size of the donation. Sorting would involve more

gas.

- **Testnet only:** The DApp is not shown on the Ethereum mainnet but on Sepolia testnet, thus, no actual money is lost.
- **No multi-token support:** The treasury only holds and transfers ETH; there is no implementation of ERC-20 token support.

11. Future Work

- **Variable m -of- n threshold:** The club should be given an opportunity to adjust the number of executives and the amount of required approvals by a governance proposal.
- **ERC-20 token support:** Have the treasury able to store and issue stablecoins (USDC, DAI) as well as ETH.
- **Governed budget cap changes:** Require 2-of-3 to change a category cap, which is a governance action.
- **Sorted donor leaderboard:** Have a sorted ranking on chain (a heap) or off-chain (The Graph) indexing.
- **Mainnet deployment:** Deploy to Ethereum mainnet with a real audit so that the system can be used by real student clubs.
- **Mobile wallet support:** Test and document WalletConnect integration for non-MetaMask wallets.
- **Event-driven UI:** Use ethers.js event listeners instead of polling to ensure that the DApp is updated immediately after each on-chain event instead of the user refreshing.

12. Conclusion

This project demonstrates how a student club can replace a single, trusted treasurer with a transparent, tamper-proof multi-signature treasury managed collectively by three elected executives. The key achievements are:

- 2-of-3 multi-sig Solidity contract where the proposer is automatically approved, so that the votes of each executive count at the very introduction of a proposal.
- Three progressive governance mechanisms on-chain: emergency pause circuit breaker, spending limits by category, and replacing executives.
- Open donor leaderboard and 7 days expiry on proposal.
- A responsive browser DApp with real-time state display, expiry countdowns, category badges, and collapsible governance panels.
- A 75-test Hardhat suite of all happy paths, access-control attacks, edge cases, time-travel expiry tests, complete end-to-end tests.
- A LaTeX technical report following the course template, with tables, figures, security analysis, and a complete step-by-step setup guide.

It can be used on the Ethereum Sepolia testnet and the Hardhat local network, and is fully functional and ready to be demonstrated live.

13. References

1. Ethereum Foundation. *Ethereum Documentation*. <https://ethereum.org/en/developers/docs/>. Accessed April 2026.
2. Ethereum Foundation. *Solidity Language Documentation v0.8.20*. <https://docs.soliditylang.org/en/v0.8.20/>. Accessed April 2026.
3. NomicFoundation. *Hardhat Documentation*. <https://hardhat.org/docs>. Accessed April 2026.
4. ethers.js Contributors. *ethers.js v6 Documentation*. <https://docs.ethers.org/v6/>. Accessed April 2026.
5. MetaMask Team. *MetaMask Developer Documentation*. <https://docs.metamask.io/>. Accessed April 2026.
6. Gnosis Safe. *Safe{Wallet} — Multi-Signature Wallet Reference Implementation*. <https://safe.global/>. Accessed April 2026.
7. ConsenSys Diligence. *Smart Contract Best Practices: Re-Entrancy*. <https://consensys.github.io/smart-contract-best-practices/attacks/reentrancy/>. Accessed April 2026.
8. OWASP. *Smart Contract Top 10*. <https://owasp.org/www-project-smart-contract-top-10/>. Accessed April 2026.
9. Buterin, V. *A Next-Generation Smart Contract and Decentralized Application Platform*. Ethereum White Paper, 2014. <https://ethereum.org/en/whitepaper/>.
10. OpenZeppelin. *OpenZeppelin Contracts Documentation — Access Control*. <https://docs.openzeppelin.com/contracts/5.x/access-control>. Accessed April 2026.

Appendix — Setup Guide for Executives

Step 1 — Install MetaMask. Each executive installs the MetaMask browser extension from <https://metamask.io> and creates a new wallet. The 12-word Secret Recovery Phrase must be written on paper and stored securely — it is the only way to recover the wallet.

Step 2 — Switch to the local Hardhat network. In MetaMask: network dropdown → Add a custom network. Enter: Name = Hardhat Local, RPC URL = <http://127.0.0.1:8545>, Chain ID = 31337, Currency = ETH.

Step 3 — Import a test account. In MetaMask: account icon → Import account → paste one of the private keys printed by `npm hardhat node` (Account #0, #1, or #2).

Step 4 — Start the local blockchain.

```
1 cd smart_gr_contract
2 npm hardhat node           # keep this terminal open
```

Step 5 — Deploy the contract.

```
1 # In a second terminal:  
2 npx hardhat run scripts/deploy.js --network localhost  
3 # Copy the printed contract address.
```

Step 6 — Update the frontend. Open `frontend/index.html`, find `const CONTRACT_ADDRESS` and replace the value with the address from Step 5. Save and open in Chrome/Firefox.

Step 7 — Run all tests.

```
1 npx hardhat test  
2 # Expected output: 75 passing
```